CS-107 Lecture 16
CPU Scheduling

Troy Astarte

Megan Venn-Wycherley

Recap: what an operating system does

- **Two categories of software:** application software addresses a user's problem; system software manages the machine itself.
- **The operating system** managesmemory, processor time and the input and output devices on behalf of every program.
- **It provides services:** program development, program execution, controlled access to files, and detection of hardware and software errors.
- **Multiprogramming** keeps several programs in memory competing for the processor, and the operating system decides which one runs.

Recap: processes and scheduling

- **Operating systems grew with the hardware:** serial processing with no operating system, then batch, multiprogrammed batch, timesharing and real-time systems.
- **A process is a program in execution,** and the operating system tracks it through states: new, ready, running, waiting and terminated.
- **Processes are created, terminated and suspended,** and a suspended process can be swapped out to disk when memory is short.
- **Queues hold the waiting processes,** with a separate queue per event so the operating system need not search one long list.
- **Each process has a process control block** holding its register values; exchanging them between processes is a context switch.

Goals for Today

- CPU scheduling: first-come first-served, shortest job next and round robin
- Memory management: logical and physical addresses, base and bounds registers
- Single contiguous memory, partitions, and how a partition is selected
- Paged memory management: pages, frames, demand paging and virtual memory

CPU Scheduling

CPU Scheduling

- The act of determining which process in the **ready** state should be moved to the **running** state
- Many processes may be in the ready state…
- But only one process can be in the running state, having it's instructions being executed on by the processor.

CPU Scheduling

- **Non pre-emptive scheduling**

The currently executing process gives up the CPU voluntarily

- **Pre-emptive scheduling**

The operating system decides to favour another process over the currently running one

CPU Scheduling Approaches

- **First-Come, First-Served**

Processes are moved to the CPU in the order in which they arrive in the running state

- **Shortest Job Next**

Process with shortest estimated running time in the ready state is moved into the running state first

- **Round Robin**

Each process runs for a specified time slice and moves from the running state to the ready state to await its next turn if not finished

Evaluating CPU Scheduling Approaches

- **Turnaround time**

Time between when a process first arrives in the ready state and when it exits the running state for the last time

- **% Delay**

Calculated by:

(turnaround - service time) / service time * 100

- **Processor Utilization**

Percentage of time CPU doing useful work

First-Come, First-Served

- Processes move to the CPU in the order they arrive in the ready state.
- Non pre-emptive: a process keeps the CPU until it finishes or blocks.
- Simple to implement, because the ready queue is first-in, first-out.
- Turnaround time depends on arrival order, so a long job at the front delays everything behind it.

First-Come, First-Served

Process Name	Arrival Time	Service Time	Start Time	Finish Time	Turnaround Time	% Delay
P1	0	140				
P2	20	75				
P3	40	320				
P4	60	280				
P5	80	125				

Shortest Job Next

- The process with the shortest estimated running time runs next.
- Non pre-emptive: a shorter job that arrives later still waits for the running process to give up the CPU.
- Gives the lowest average turnaround time of the three, when the estimates are accurate.
- Needs an estimate of service time, and a long process can be postponed repeatedly.

Shortest Job Next

Process Name	Arrival Time	Service Time	Start Time	Finish Time	Turnaround Time	% Delay
P1	0	140				
P2	20	75				
P3	40	320				
P4	60	280				
P5	80	125				

Round Robin

- Each process gets the CPU for a fixed time slice, then goes to the back of the ready queue.
- Pre-emptive: the operating system interrupts the running process when its slice expires.
- The slice length is the trade-off: too short wastes time switching, too long behaves like first-come, first-served.

Round Robin (with time slice of 100)

Process Name	Arrival Time	Service Time	Start Time	Pause	Restart	Pause	Restart	Pause	Restart	Finish Time
P1	0	140								
P2	20	75								
P3	40	320								
P4	60	280								
P5	80	125								

Round Robin (with time slice of 100)

Process Name	Arrival Time	Service Time	Start Time	Finish Time	Turnaround Time	% Delay
P1	0	140	0	515		
P2	20	75	100	175		
P3	40	320	175	940		
P4	60	280	275	920		
P5	80	125	375	740		

Comparing the three approaches

Approach	How the next process is chosen	Pre-empts?	Strength	Weakness
First-Come, First-Served	Order of arrival in the ready state	No	Simple, and no process is ever skipped	One long job delays every process behind it
Shortest Job Next	Shortest estimated running time first	No	Lowest average turnaround time of the three	Needs an estimate of run time; long jobs may wait indefinitely
Round Robin	Each process in turn, for a fixed time slice	Yes	Shares the CPU evenly, so interactive users get a response	Context switches cost time; the slice length has to be tuned

Memory Management

Memory Management

- Operating systems must employ techniques to track where and how a program resides in memory.
- Achieved by converting logical addresses into physical addresses
- **Logical address**

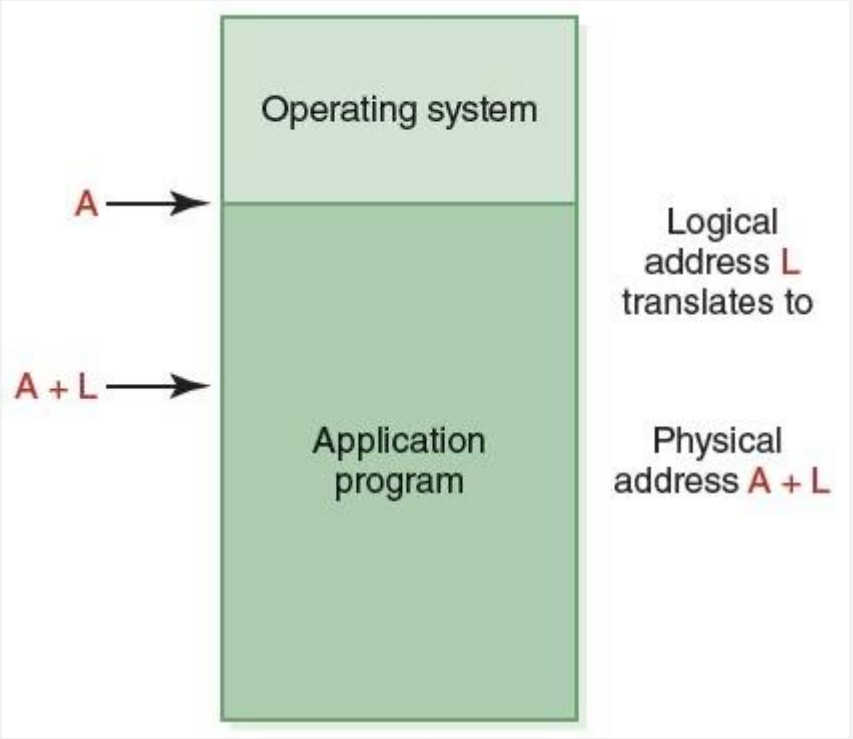
Reference to a stored value **relative** to the program making the reference

- **Physical address**

Actual address in main memory, where the instruction/data is stored

Addressing

- A **base register** points to the starting point of the program. E.g. A in the diagram.
- A **logical address** is simply an integer value relative to the starting point of the program. E.g. L in the diagram.
- A **physical address** is a logical address added to the starting location of the program in main memory. E.g. A + L



Memory Management

- **Responsibilities**:
  - Process isolation
  - Automatic allocation and management
  - Support of modular programming
  - Protection and access control
  - Long-term storage

- We need to keep a record of where our processes are in memory (either main or secondary), remember back to the **Process Control Block** from last lecture.

Single Contiguous Memory

- One approach is **single contiguous memory management**

- There are only two programs in memory
  - The operating system
  - The application program

- Very limited capabilities. Where might this be a useful approach?



Embedded systems, real-time systems, cheap hardware…

What is a partition?

- **A partition is a block of memory given to one program.** The operating system and several programs sit in memory at the same time, each in its own partition.
- **Each partition has a base and a bounds register,** so every address is translated and checked while the program runs.
- **Both schemes waste memory:** fixed partitions leave unused space inside a partition, dynamic partitions leave gaps between programs as they finish.

Partition Memory Management

- **Single contiguous MM** has only OS and one other program in memory at one time
- **Partition MM** has the OS and any number of other programs in memory at one time
- Two schemes for dividing up memory for programs:
  - **Fixed partitions:** memory is divided in advance into a set number of partitions, and a program is loaded into one large enough to hold it.
  - **Dynamic partitions:** a partition is created to fit each program as it is loaded, so the space inside it is not wasted.

Partition Memory Management

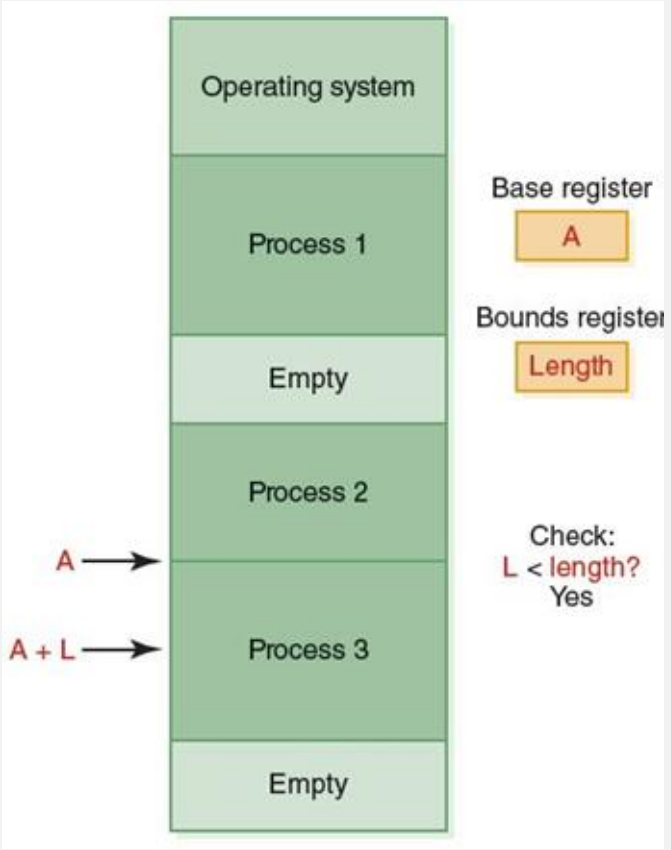
Memory is divided into a set of partitions, some empty and some allocated to programs

- **Base register**

A register that holds the beginning address of the current partition (the one that is running)

- **Bounds/Limit register**

A register that holds the length of the current partition.



Partition Selection

- In order to load a program into memory, we want to have space to load its instructions. (ish, more on this later)
- In a fixed partition scheme, we have our memory separated into fixed-size partitions. The partitions may be different sizes, but they are unable to change size (hence fixed).
- We receive requests to load programs into memory and check the request against a list of partitions, hopefully finding a partition which is free and large enough to hold the program

Partition Selection

- In the next slides, similar to the CPU scheduling approaches, we will look at some methods to finding partitions in which to place a program.
- To do this we will have a list of requests and their sizes, as well as memory partitioned into various fixed-size partitions.
- We will then use one of the approaches to allocate requests to suitable partitions.

Partition Selection Approaches

Partition Selection Approaches

- **Best fit**

Allocated program to the smallest partition big enough to hold it

- **First fit**

Allocate program to the first partition big enough to hold it

- **Next fit**

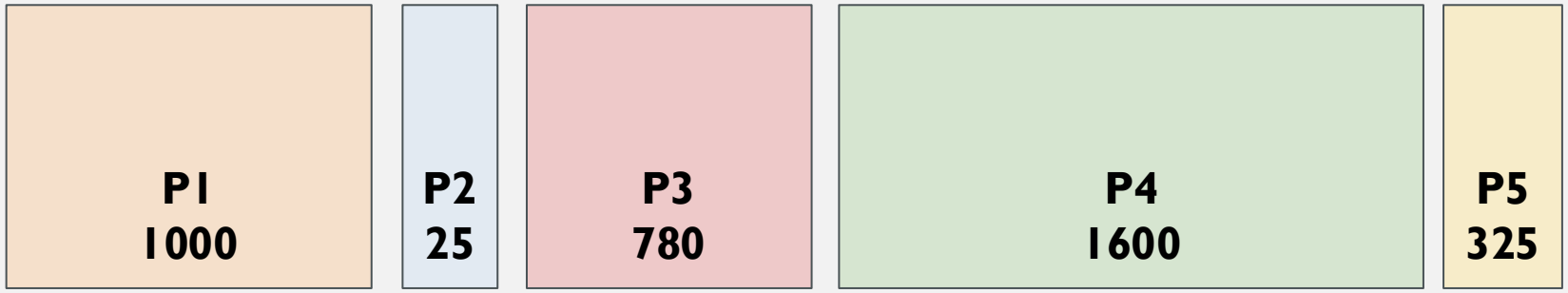
Allocate program to next partition big enough to hold it

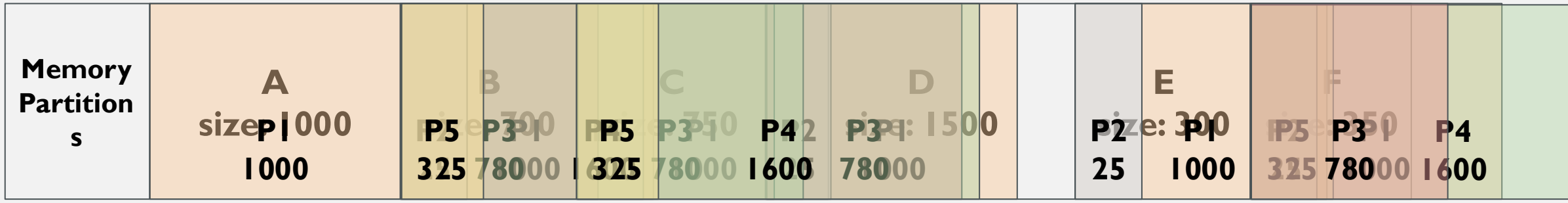
- **Worst fit**

Allocate program to the largest partition big enough to hold it

Partition Selection: Best Fit

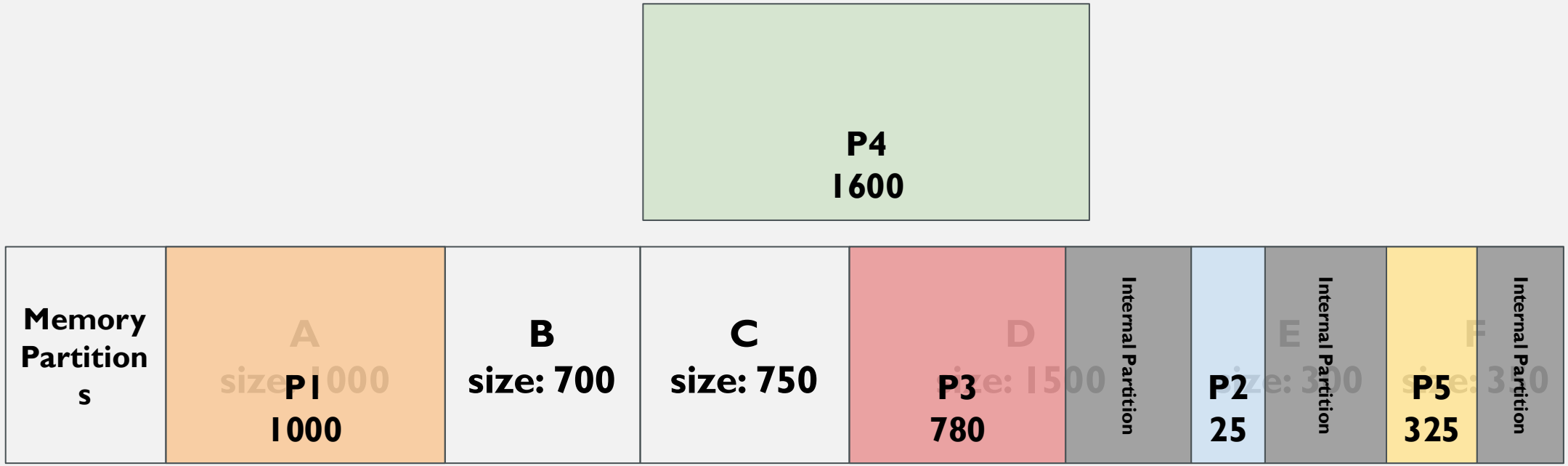
Requests come in for blocks of the following sizes (in order):
1000, 25, 780, 1600, and 325





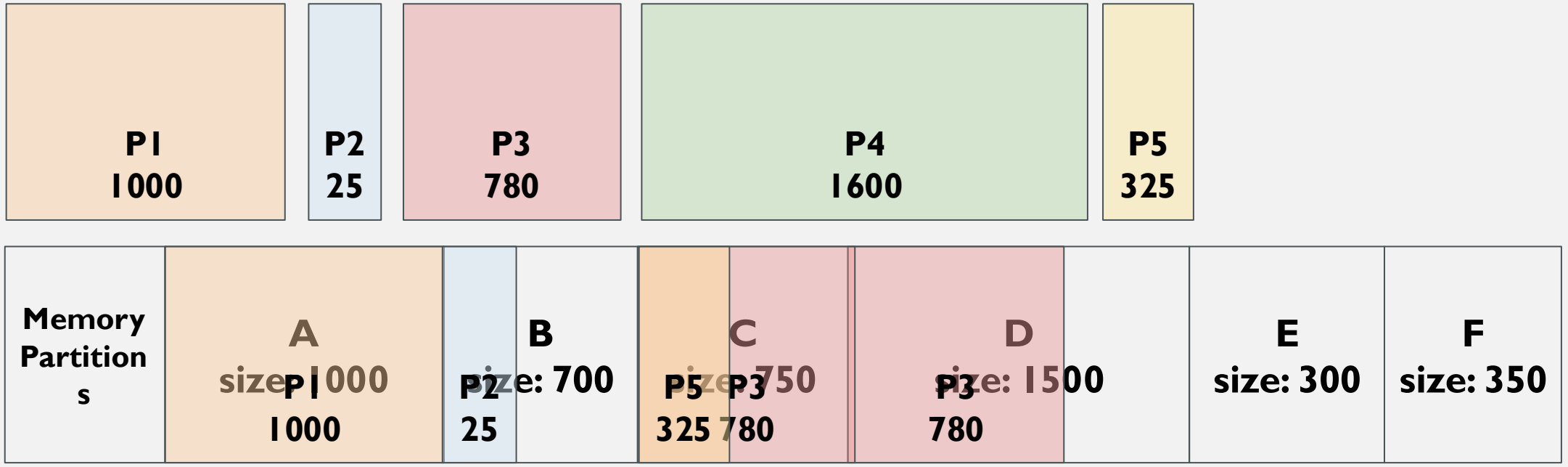
Partition Selection: Best Fit

Requests come in for blocks of the following sizes (in order):
1000, 25, 780, 1600, and 325



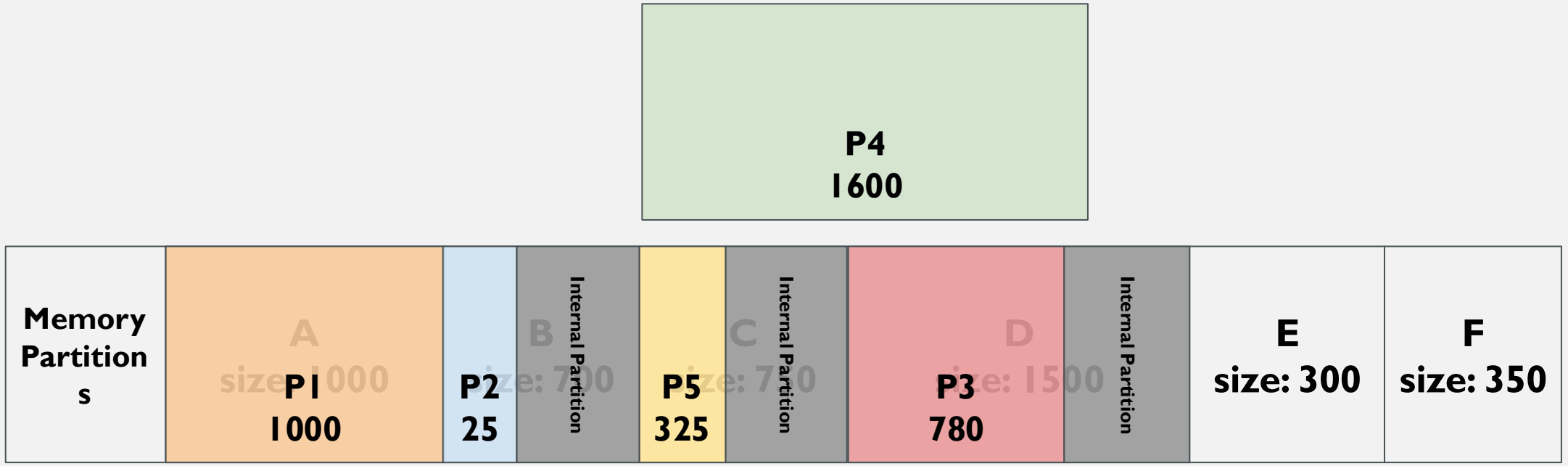
Partition Selection: Best Fit

Requests come in for blocks of the following sizes (in order):
1000, 25, 780, 1600, and 325



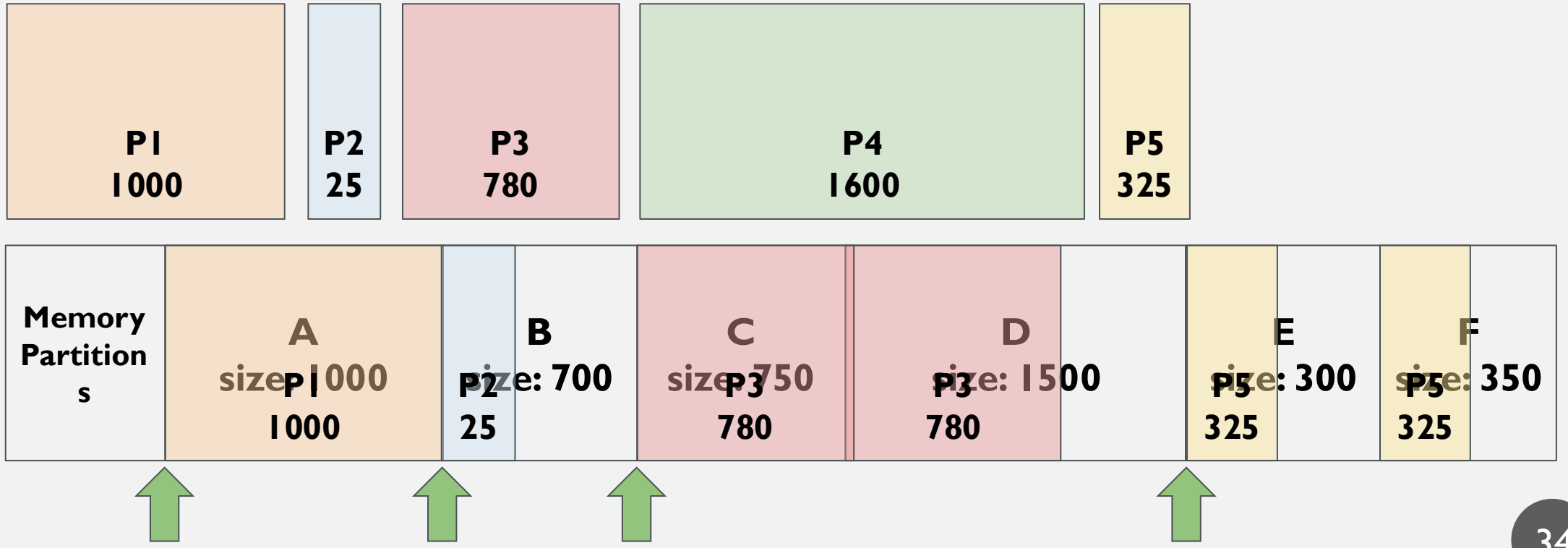
Partition Selection: Best Fit

Requests come in for blocks of the following sizes (in order):
1000, 25, 780, 1600, and 325



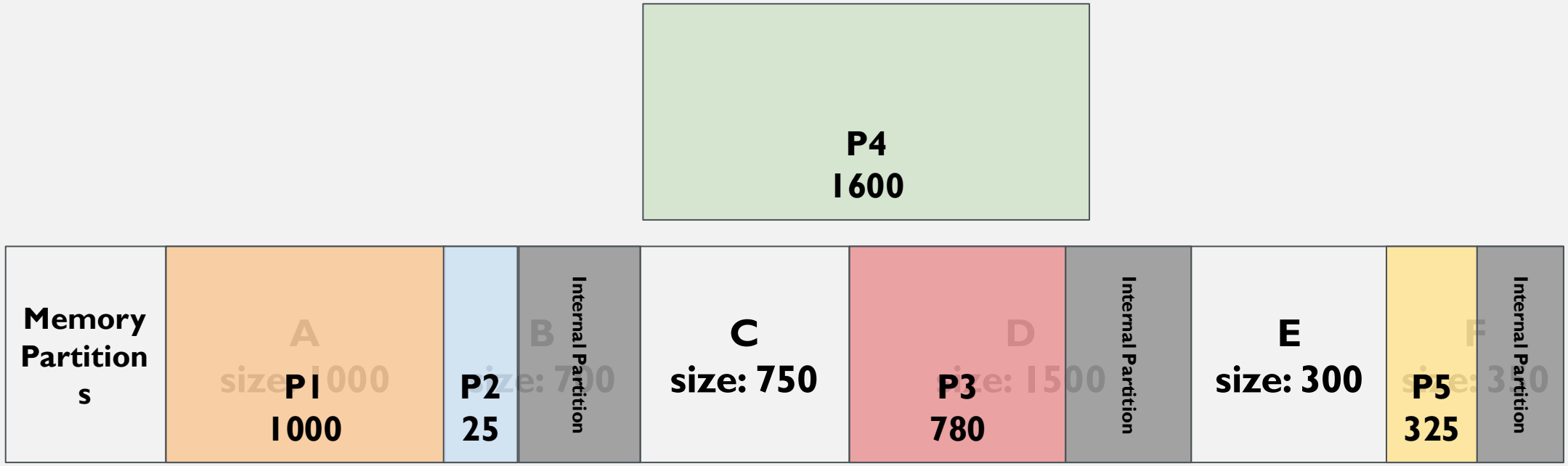
Partition Selection: Next Fit

Requests come in for blocks of the following sizes (in order):
1000, 25, 780, 1600, and 325



Partition Selection: Next Fit

Requests come in for blocks of the following sizes (in order):
1000, 25, 780, 1600, and 325



Partition Selection Issues

- **Best Fit**

Fragmentation occurs and long search times are needed

- **First Fit**

Fragmentation due to smaller partitions at head of the list. Search time creeps up

- **Next Fit**

Fragmentation occurs, but worst effects are delayed as more distributed in memory

- **Worst Fit**

Long search times. Reduction of potential large partitions, but some programs may be impossible to load

Paged Memory Management

What a page is

- A **page** is a fixed-size block of a process, and every page is the same size, commonly 4 KB.
- A **frame** is a fixed-size block of main memory of that same size, so any page fits in any frame.
- The operating system decides where the boundaries fall (not the programmer), so a page can end part way through an array or a routine .
- The pages of one process need not be next to each other, or in order, so memory freed by finished processes can be reused.
- Since pages and frames are identical in size, no gaps are left between them. The only waste is the unused remainder of a process's last page.

Paged Memory Management

- **Demand paging**

An extension of paged memory management in which pages are brought into memory on demand

- **Page swap**

The act of bringing in a page from secondary memory, which often causes another page to be written back to secondary memory

Paged Memory Management

- **Virtual memory**

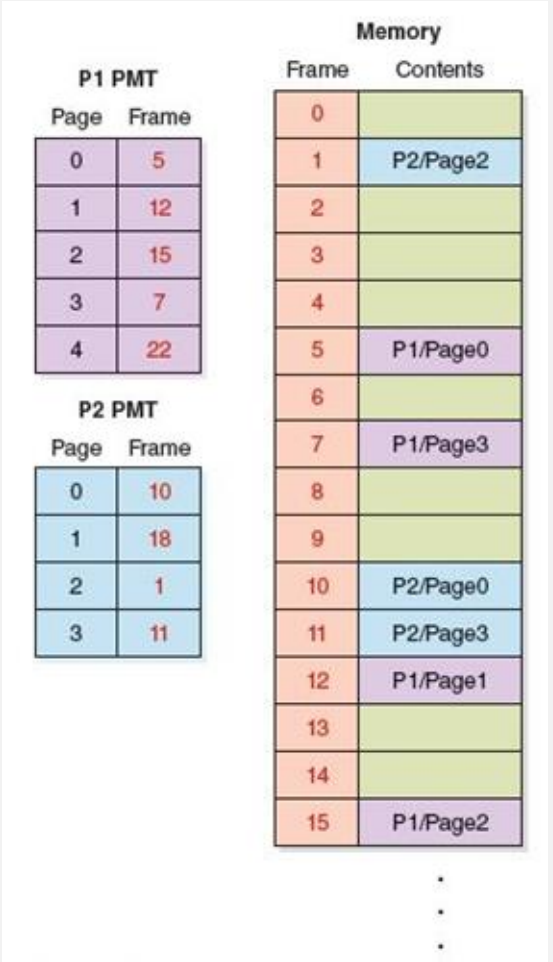
The illusion that there are no restrictions on the size of a program because an entire process doesn't have to be in memory at the same time

- **Thrashing**

Inefficient processing caused by constant page swaps

Paged Memory Management

- A **page management table (PMT)** references where in memory each page of the program is stored in.
- This lets the processes be loaded into memory in a **non-contiguous** way that is both fast and effective when using a multiprogramming system.
- **Non-contiguous** means the pages of one process sit in any free frames, not in one unbroken block and not in page order. Page 0 might be in frame 5 and page 1 in frame 12, as in the table.



Goals for Today

- CPU scheduling: first-come first-served, shortest job next and round robin
- Memory management: logical and physical addresses, base and bounds registers
- Single contiguous memory, partitions, and how a partition is selected
- Paged memory management: pages, frames, demand paging and virtual memory

CS-107 Lecture 17
Files, Directories and Systems

Troy Astarte

Megan Venn-Wycherley

Goals for Today

- File types, the operations we perform on files, and who is allowed to perform them
- Sequential and direct access, and file protection as it works in Unix
- Directories and directory trees, on Windows and on Unix
- Working directories, and absolute against relative paths
- Three ways to store a file on disk: contiguous, chained and indexed allocation

File Systems

File Systems

- **File**

Named collection of related data, to organise secondary memory

- **File system**

The operating system's logical view of the files it manages

- **Directory**

A named group of files

What the operating system stores about a file (some…)

- Name: the label people use. The disk itself knows nothing about it.
- Type, and on some systems the extension that suggests which program opens it.
- Size in bytes, and the number of blocks the file occupies.
- Location: where the data sits on disk, held as a start and a length, a chain of pointers, or an index.
- Owner and permissions: who may read, write or run it.
- Timestamps: when it was created, last changed and last read.

Types of Files

- **Text file**

File that contains characters from the ASCII or Unicode character sets

- **Binary file**

File that contains data in a specific format, requiring special interpretation of its bits

Types of Files

The terms text file and binary file are somewhat misleading

Ultimately, all information on a computer is stored as binary digits

- **Text files** are formatted as chunks of 8 or 16 bits, interpreted as characters
- **Binary files** are formatted in some other special format

File Types

- **File Type**

The kind of information contained in a document
Most files, regardless of format, contain a specific type of information

- **File extension**

Part of a file name that indicates the type

File names are often in two parts:

File_name . File_Extension

File Types

Extensions	File type
txt	text data file
mp3, au, wav	audio file
gif, tiff, jpg	image file
doc, wp3	word processing document
java, c, cpp	program source files

File Operations

- Often we will want to carry out basic operations on our files:
  - Create
  - Read
  - Write
  - Delete
- We may also want to do higher level operations such as:
  - Moving / Copying
  - Changing access permissions
  - Renaming / Changing meta information

File Access

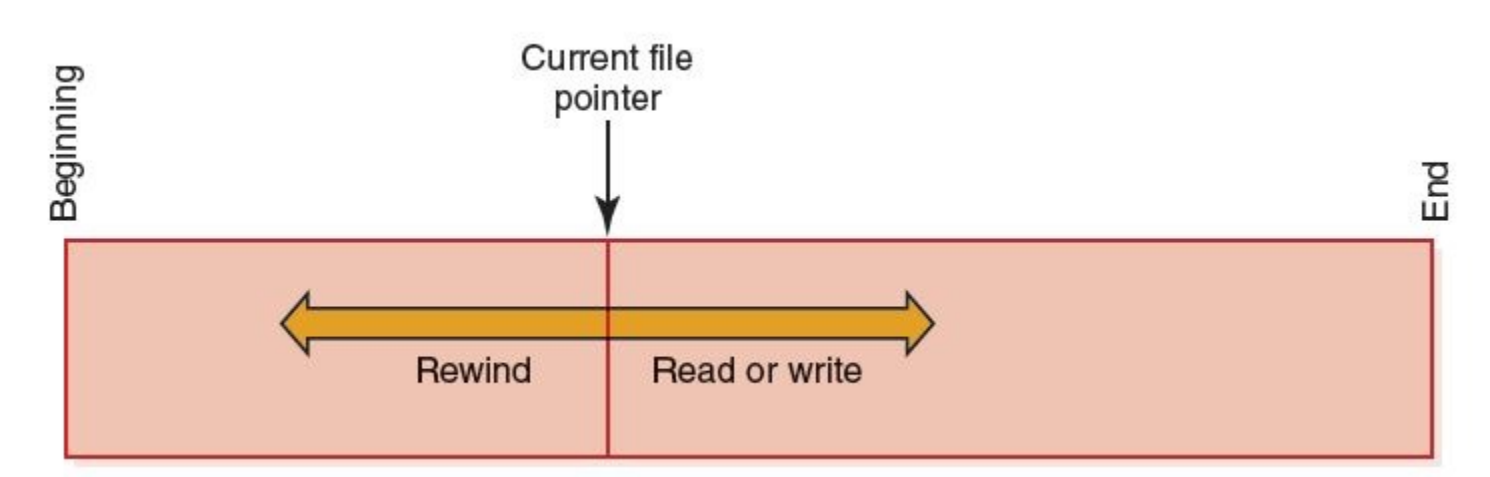
- **Sequential access**

The technique in which data in a file is accessed in a **linear fashion**; to get the last record, you must read all records

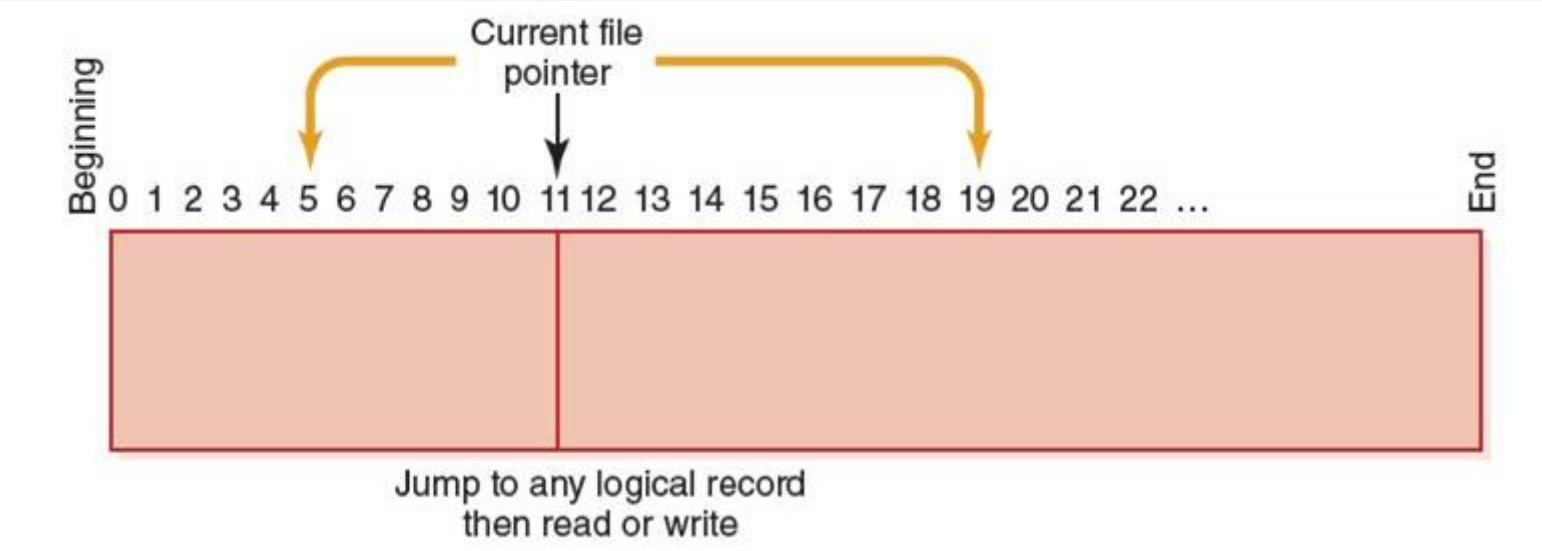
- **Direct access**

The technique in which data in a file is conceptually divided into numbered logical records and **accessed directly**, by specifying logical record numbers

Sequential Access



Direct Access



File Protection

- The process of limiting file access
- In multiuser systems, file protection is of primary importance
- We don't want one user to be able to access another user's files unless the access is specifically allowed
- A file protection mechanism determines who can use a file and for what general purpose

File Protection

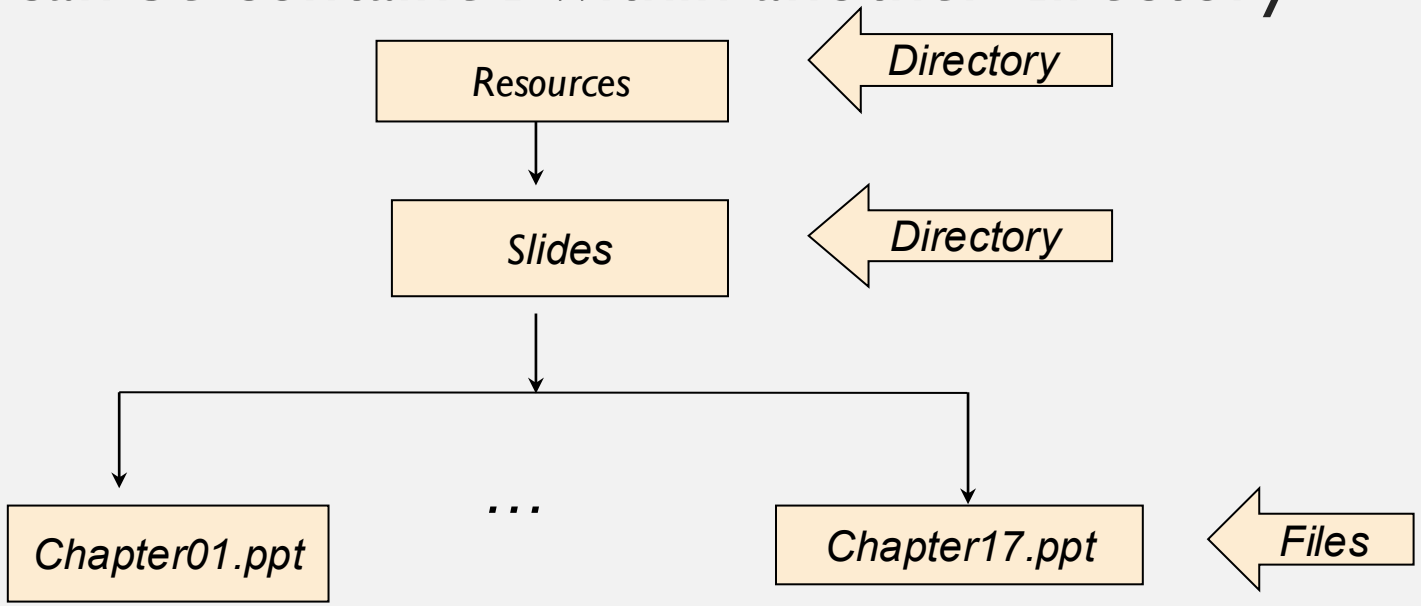
An example of a file protection scheme is the file settings in the Unix operating system, which are divided into three categories

	Read	Write/Delete	Execute
Owner	Yes	Yes	No
Group	Yes	No	No
World	No	No	No

Directories

Directory Trees

- Recall that a directory is a named group of files.
- A directory can be contained within another directory



Directory Trees

- **Directory tree**

A logical view of a file system; a structure showing the nested directory organization of a file system

- **Root directory**

The directory at the highest level

- **Parent directory**

The containing directory

- **Subdirectory**

The directory being contained

Directory Trees

- An example from a Windows system:

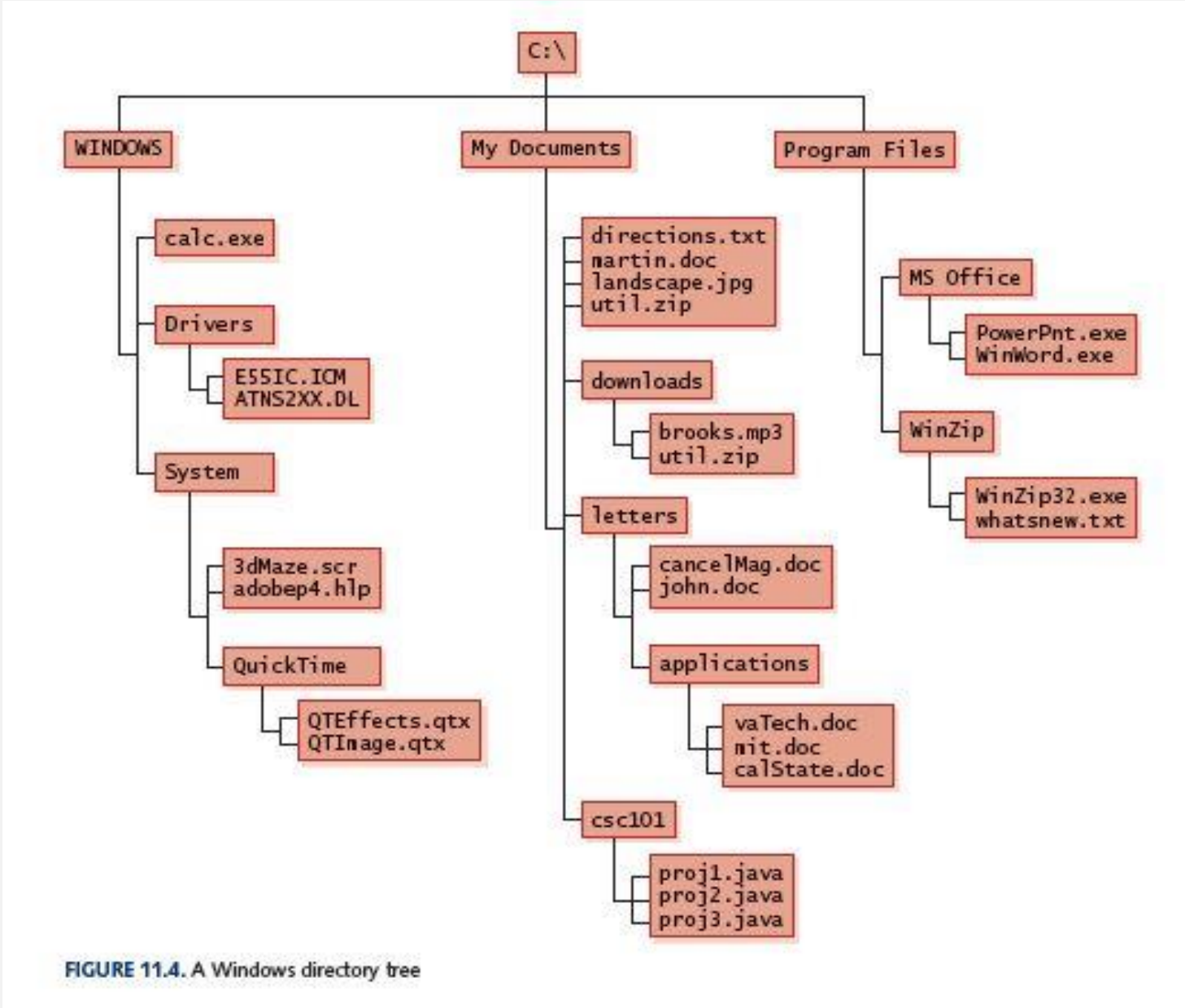


FIGURE 11.4. A Windows directory tree

Directory Trees

- An example from a UNIX system:

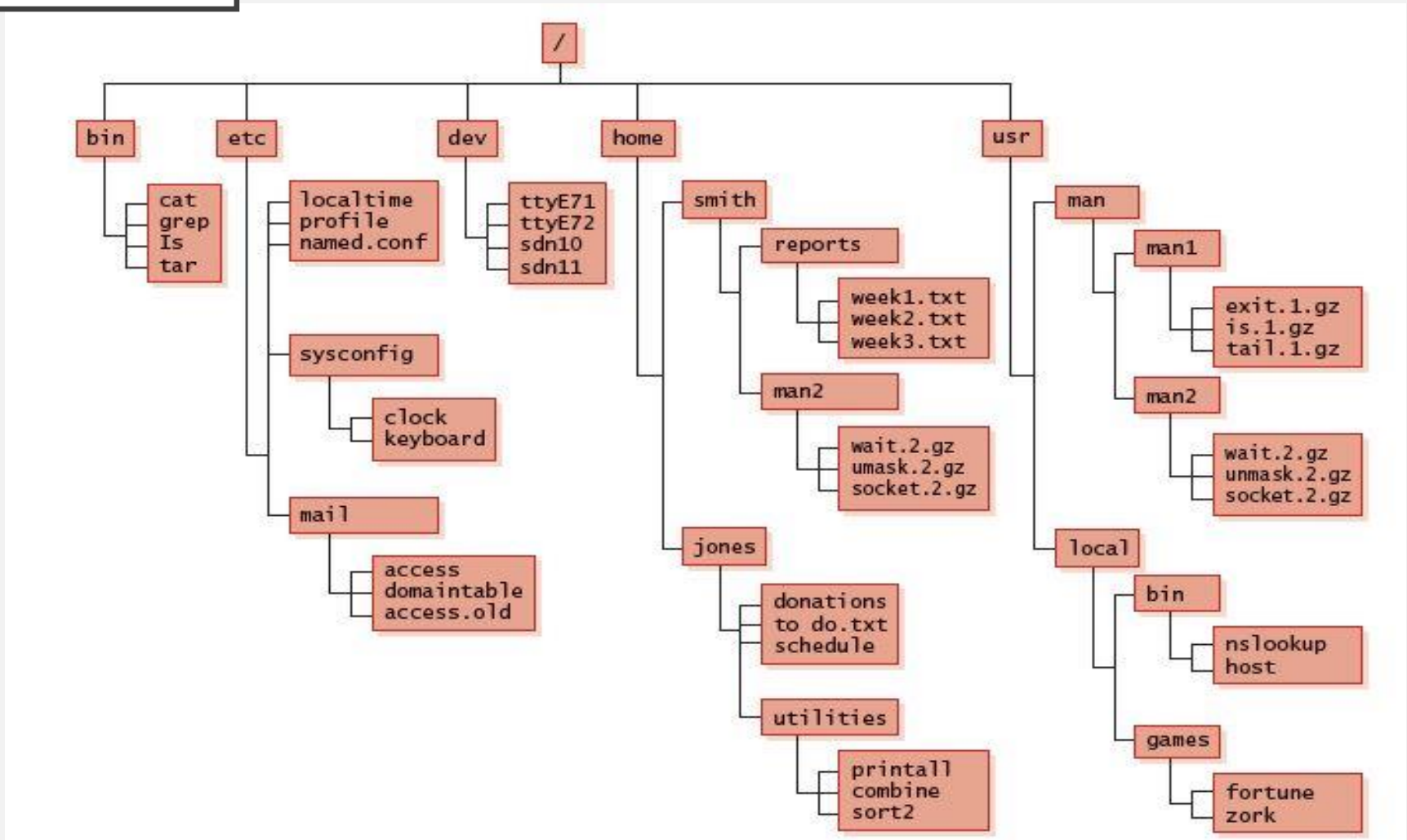


FIGURE 11.5 A UNIX directory tree

Directory Trees

- Each directory in the tree should have a reference to it's parents.
- We also say that a directory has a reference to itself.
- A directory's parents is referenced by a pointer often shorthanded to ".." (two dots)
- The reference to itself is referenced by a pointer shorthanded to "." (a single dot).

Paths

At any point in time, you can be thought of as working in a particular location (a particular subdirectory)

- **Working directory**

The subdirectory in which you are working

- **Path**

A text designation of the location of a file or subdirectory in a file system. A route to take to get to a file or subdirectory.

Paths

- **Absolute path**

A path that begins at the root and includes all successive subdirectories

- **Relative path**

A path name that begins at the current working directory

Path Examples

Paths to “proj1.java”:

Absolute path:
C:\My Documents\csc101\proj1.java

Relative path:
If we are currently in “C:\My Documents\letters” then the relative would be “..\csc101\proj1.java”

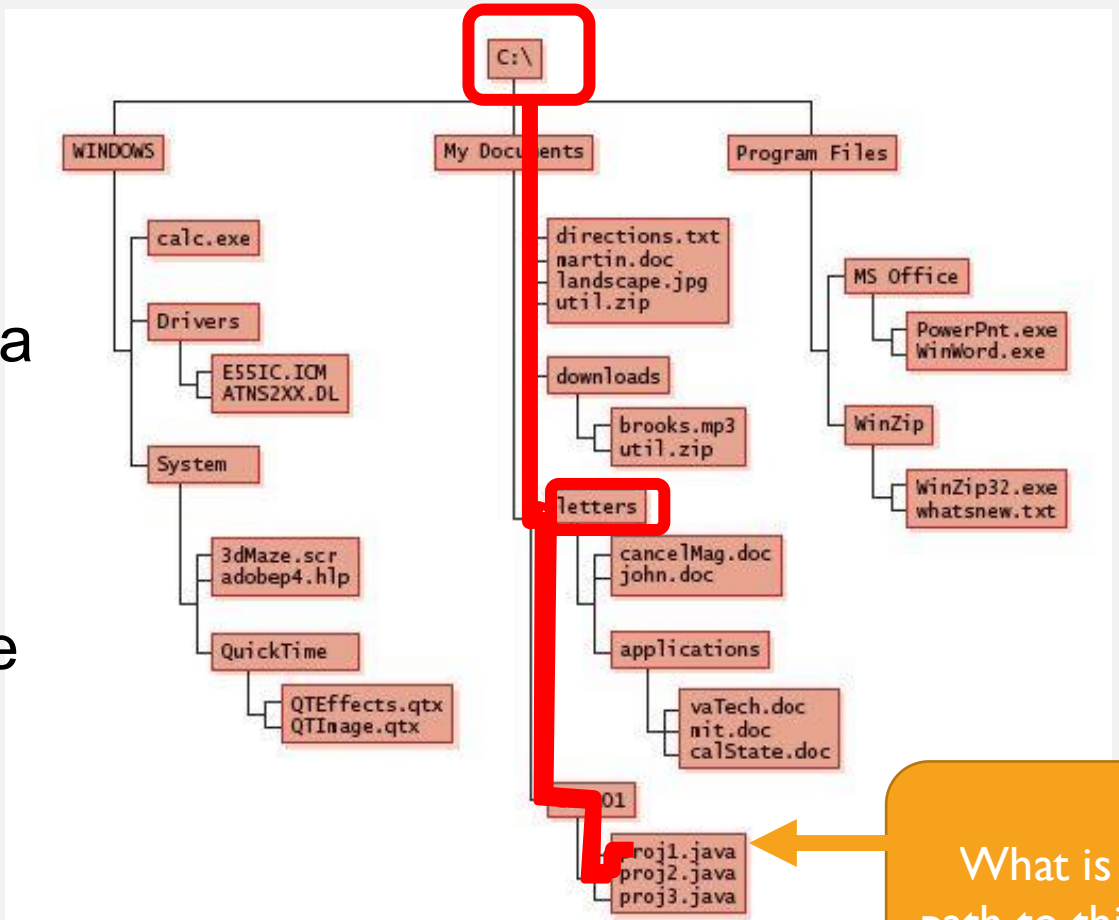


FIGURE 11.4. A Windows directory tree

What is the path to this file?

Path Examples

Absolute path:

C:\WINDOWS\System\3dMaze.scr

Relative path from WINDOWS?:

"..\System\3dMaze.scr"

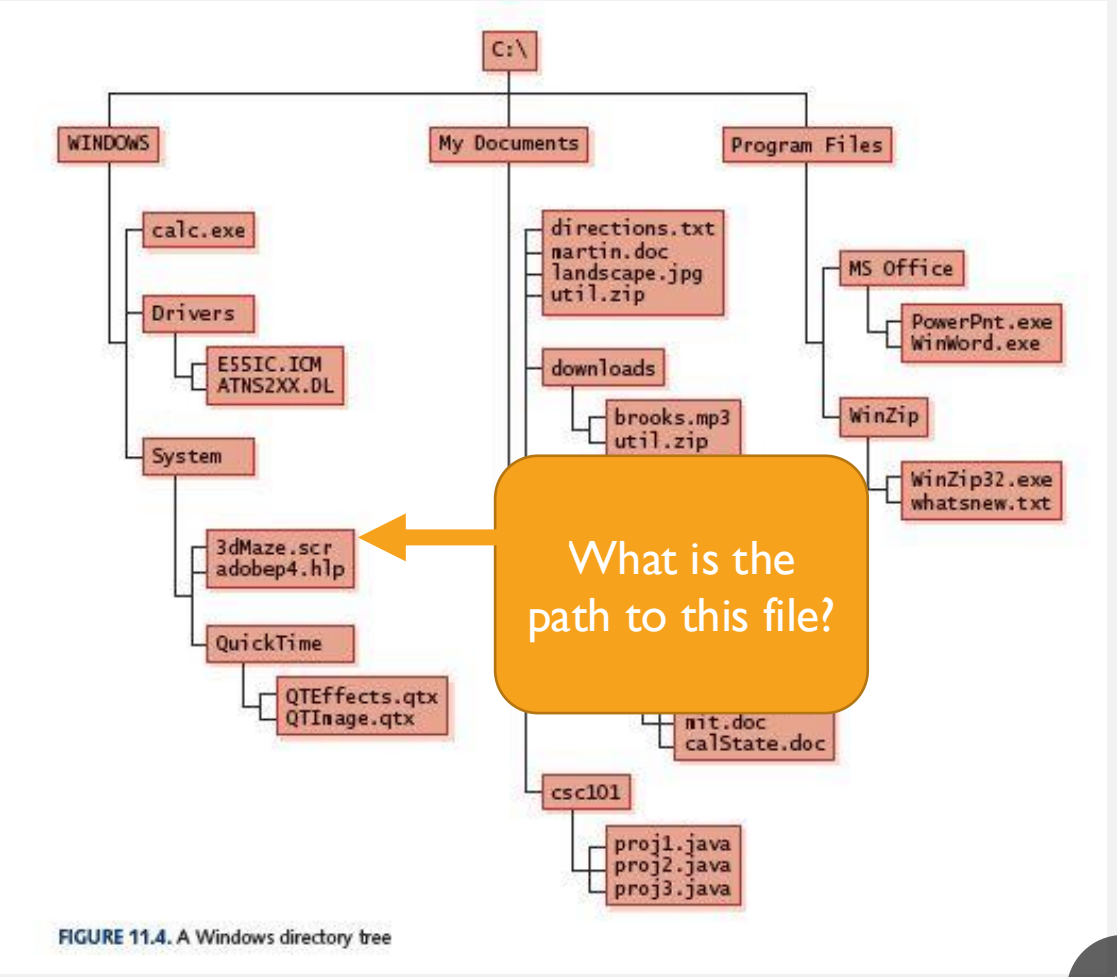


FIGURE 11.4. A Windows directory tree

File Systems and Approaches

File Systems

- It is important to note that the majority of files are small and only a few files are large.
- This makes it complicated to store files in an efficient way, as we may have lots of space, but just not necessarily as one contiguous unit.
- An operating system may choose to chunk up files and store them in various areas on the storage media (HDD, SSD etc.)

File System Approaches

- **Contiguous**

File is stored as a collection of adjacent blocks

- **Chained**

File stored in potentially non-contiguous blocks, with pointers linking the blocks together as they are populated with data

- **Indexed**

File stored in potentially non-contiguous blocks, but pointers to the blocks are defined up front in an allocation table

Free space: how the disk knows what is spare

- All three approaches need a record of which blocks are not in use - A free list chains the unused blocks together, each pointing to the next, and allocation takes them from the front.
- Remember Run Length Encoding? A bitmap keeps one bit per block, set for used and clear for free, so the state of the whole disk fits in a small table.
- Finding a run of adjacent free blocks is quick to check in a bitmap and slow in a free list, which is what contiguous allocation needs.
- Deleting a file does not erase anything: it marks those blocks free in this record, which is why deleted files can sometimes be recovered!

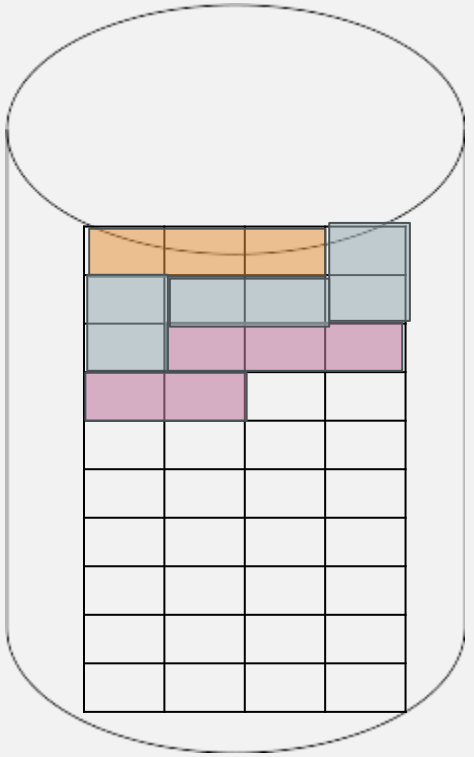
Contiguous

A disk is divided into blocks of equal size. A file has to be stored in some number of them.

Contiguous allocation puts the whole file in one unbroken run of blocks, one after another.

So the directory needs only two numbers per file: the block it starts at, and how many blocks long it is.

The operating system keeps a list of free blocks, and must find a gap long enough before it can write the file.



Name	Start	Length
File1	0	3
File2	9	5

Contiguous: the trade-off

Advantages:

Reading the file in order is as fast as the disk allows, because the blocks are neighbours and the head hardly moves.

Jumping to any point is simple arithmetic: block n of the file is the start block plus n.

Disadvantages:

Deleting files leaves gaps. Free blocks exist, but not enough in a row, so a new file is refused. This is external fragmentation.

The size of the file must be known before it is written, and a file that grows has to be copied to a longer gap.

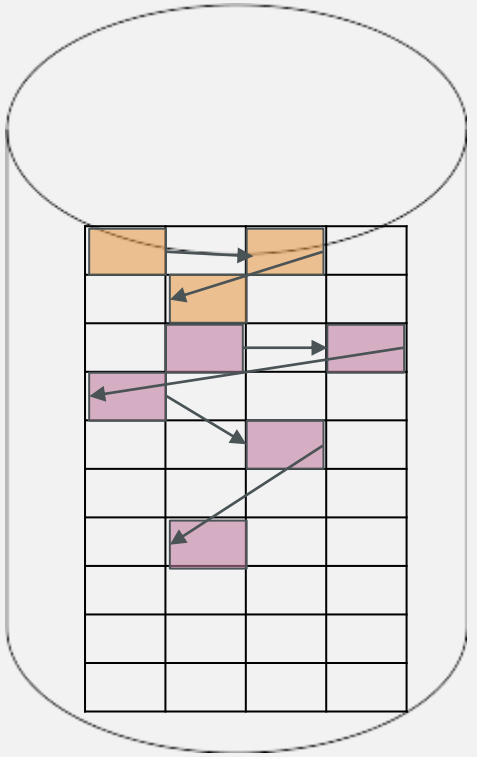
Chained

Chained allocation lets the blocks of one file sit anywhere on the disk.

Each block stores a small pointer giving the address of the next block in the file (you'll learn more about this in programming!)

The directory holds only the first block, and usually the last, so reading a file means following the chain from the front.

The last block's pointer is empty, which is how the system knows the file has ended.



Name	Start	Length
File1	0	3
File2	9	5

Chained: the trade-off

Advantages:

Any free block will do, so no space is wasted waiting for a long enough gap. There is no external fragmentation.

A file can grow whenever it needs to: take a free block and link it on the end. Its size need not be known in advance.

Disadvantages:

Reaching block 20 means reading the 19 blocks before it, so jumping about in a file is slow.

Part of every block goes to its pointer, the blocks are scattered, so the disk head moves more, and one damaged pointer loses the rest of the file.

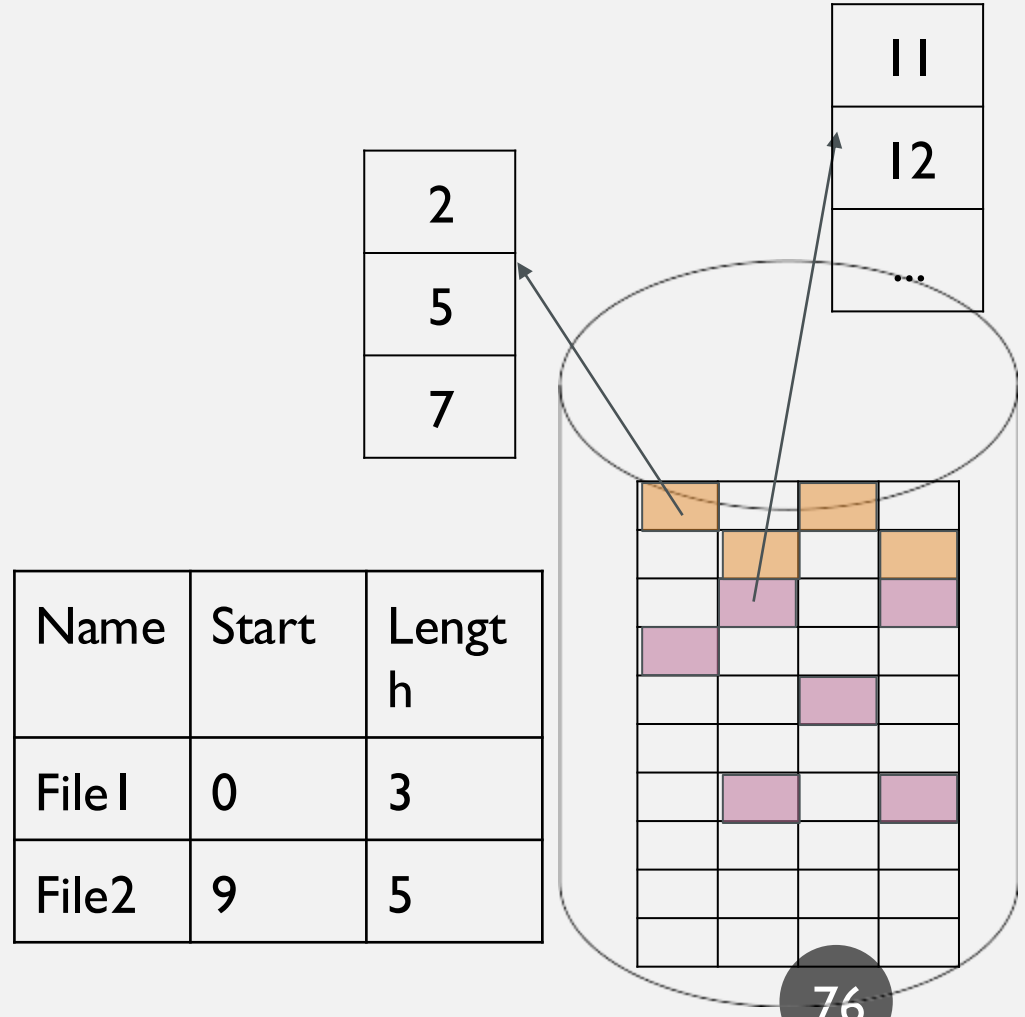
Indexed

Indexed allocation keeps all the addresses of one file together, in an index block.

The index block is created with the file and holds a list of pointers, one for each block of data, in file order.

The directory points at the index block, and the data blocks themselves can sit anywhere on the disk.

To read the fourth block of the file, the system looks at the fourth entry in the index and goes straight there.



Name	Start	Length
File1	0	3
File2	9	5

Indexed

When the file is created, an index block is allocated to hold an array of pointers, one for each data block. The directory points at the index block, and the data blocks themselves can sit anywhere.

Advantages:

- Random access is fast: one look-up in the index gives the block, with no chain to follow.
- No external fragmentation, because the data blocks need not be adjacent.
- A file can grow while the index still has spare entries.

Disadvantages:

- Every read costs an extra access, to the index block first and then the data block.
- A small file still needs a whole index block, most of which is unused.
- The number of pointers caps the file size, unless one index block can link to another.

Comparing the three approaches

Approach	Reading in order	Jumping to a point	Wasted space	Growing a file
Contiguous	Fastest: the blocks are neighbours	Instant: start block plus n	Gaps between files that are too short to reuse	Must be copied to a longer gap
Chained	Good, but the blocks are scattered	Slow: follow the chain from the front	A pointer inside every block	Link on one more free block
Indexed	Good: the index gives every address	One look-up in the index	A whole index block, even for a tiny file	While the index has spare entries

How real systems do it

- FAT, used on USB sticks and memory cards, is chaining, with the pointers pulled out of the blocks into one file allocation table.
- Unix and Linux use indexing: every file has an inode holding its details and the addresses of its blocks.
- NTFS on Windows records every file in a master file table, and stores very small files inside their own record.
- No mainstream system uses contiguous allocation for ordinary files, but CDs and DVDs do, because nothing on them is ever rewritten.

Goals for Today

- File types, the operations we perform on files, and who is allowed to perform them
- Sequential and direct access, and file protection as it works in Unix
- Directories and directory trees, on Windows and on Unix
- Working directories, and absolute against relative paths
- Three ways to store a file on disk: contiguous, chained and indexed allocation

CS-107 Lecture 17
Operating Systems: Files, Directories and Systems

Troy Astarte

Megan Venn-Wycherley